TurboRUL

AI predictive maintenance for aircraft engines — progress report
Phases 1–3 of 6 complete · 12 October 2026

1. Summary

TurboRUL predicts the **Remaining Useful Life (RUL)** of turbofan engines, in flight cycles, from sensor histories, and will turn those predictions into a ranked maintenance work list. It uses NASA's public C-MAPSS run-to-failure dataset. Phases 1–3 are done: a tested data pipeline, four model families (linear, XGBoost, LSTM, 1D-CNN) tuned with Optuna on validation engines only, and a first scoring of the official FD001 test set, averaged over five seeds.

- **All three tuned models meet the RMSE target (≤ 15 cycles).** Best: XGBoost at 14.19 ± 0.23 cycles; 1D-CNN 14.90 ± 0.55. The stretch target (≤ 13) is not met.
- **Near failure the models are precise.** For engines with true RUL ≤ 30, error is 3.6–5.3 cycles (linear: 11.5), and no truly critical engine is ever classed Healthy.
- **Critical-engine recall:** 1D-CNN reaches 92% (target 90%); the others reach 84% and 82%.
- **Deep models do not beat XGBoost on FD001.** The gap (0.7 cycles) is about one seed-to-seed standard deviation; Section 4.4 explains why.
- **Tuning improved validation scores but barely moved test scores.** With only 20 validation engines, model selection is noisy. Engine-level cross-validation is planned for Phase 4.
- **Improvement over the linear baseline is 14%**, short of the 25% target, because the baseline already uses engineered trend features.

2. Objectives and status

Success metric (PRD §2)	Target	Status after Phase 3
RMSE on FD001 test set	≤ 15 cycles (stretch ≤ 13)	**Met.** XGBoost 14.19, 1D-CNN 14.90, LSTM 14.99. Stretch not met.
NASA asymmetric score	Reported; beats own baselines	**Partly.** XGBoost 420, LSTM 306, 1D-CNN 394 vs linear 413; LSTM and 1D-CNN beat it.
Improvement over linear baseline	≥ 25% lower RMSE	**Not met.** 14% (best tuned model); 17% for default XGBoost.
Critical-engine recall (true RUL ≤ 30)	≥ 90%	**Partly.** 1D-CNN 92%, XGBoost 84%, LSTM 82%.
Results on all four subsets	RMSE for FD001–FD004	**Pending** (Phase 4). Per-condition normalisation is built and tested.
Demo latency	Prediction in < 2 s	**Pending** (Phase 5). Model inference already takes milliseconds.

3. Data and method

C-MAPSS simulates a fleet of commercial turbofans run cycle by cycle until a health margin crosses a failure threshold; each cycle records 3 operating settings and 21 noisy sensors. Training files hold full run-to-failure histories; test files stop before failure, with the true RUL at the cut-off supplied separately. Counts below are measured from the files. All results in this report are on **FD001**, the standard primary benchmark.

Subset	Train engines	Test engines	Operating conditions	Fault modes	Life (min / median / max)	Test histories < 30 cycles	Test RUL ≤ 30
FD001	100	100	1	HPC	128 / 199 / 362	0	25
FD002	260	259	6	HPC	128 / 199 / 378	6	61
FD003	100	100	1	HPC + fan	145 / 220 / 525	0	20
FD004	249	248	6	HPC + fan	128 / 234 / 543	11	53

3.1 Pipeline

Step	What happens	Leakage guard
Split	Training engines split by engine ID: 80% train, 20% validation (FR-3).	Unit test: no engine on both sides.
Labels	RUL = last cycle − cycle, clipped at the RUL cap (default 125, tuned per model).	Unit-tested countdown and cap.
Preprocess	k-means operating conditions (6 for FD002/FD004), z-score each sensor within its condition, keep sensors with trend strength ≥ 0.2 (14 of 21 on FD001) (FR-2).	Fitted on train engines only.
Windows	Overlapping windows of 20–40 cycles; label = RUL at the window's last cycle; short histories front-padded.	Unit test: windows never cross engines.
Features	Classical models: mean, std, slope, last, min, max per sensor per window (84 features at 14 sensors).	—
Models	Linear regression; XGBoost (early stopping); LSTM and 1D-CNN in PyTorch (Adam, MSE, early stopping) (FR-4, FR-5).	Early stopping on validation engines.
Tuning	Optuna TPE, 40/25/25 trials (XGBoost/LSTM/CNN), median pruning; window length and RUL cap tuned with model parameters.	Test set never loaded.
Evaluate	Each test engine scored on its last window vs RUL_FD001.txt: RMSE, NASA score, critical recall (FR-6). 5 seeds; mean ± std.	Final configuration chosen on validation.

Test-like validation. Early stopping uses every window of the validation engines, but tuning and model selection use a set built to mirror the test protocol: each validation engine is cut at 10 random points (true RUL ≤ 150), and each cut is scored on its last window against the true, uncapped RUL. This lets Optuna compare different RUL caps fairly, which capped validation labels cannot do.

Selection rule. Each family was trained with default and tuned parameters. The configuration reported as *final* is the one with the lower mean validation RMSE. This is the standard rule, applied uniformly, and it was not revised after the test scores were seen. Test scores for the non-final configurations are shown only to make the effect of tuning visible.

4. Results on FD001

4.1 Model comparison

Model	Params	Window / cap	Val RMSE	Test RMSE	NASA score	Critical recall	Size	Train time
Linear regression	default ✓ final	30 / 125	19.82	**16.47**	413	0.60	84 coefs	0 s
XGBoost	default	30 / 125	17.63	13.60 ± 0.14	269 ± 10	0.81 ± 0.05	102 trees	1 s
XGBoost	tuned ✓ final	40 / 135	14.99	**14.19 ± 0.23**	420 ± 35	0.84 ± 0.04	117 trees	2 s
LSTM	default	30 / 125	17.37	14.91 ± 0.28	329 ± 42	0.86 ± 0.05	56k params	36 s
LSTM	tuned ✓ final	40 / 125	15.29	**14.99 ± 0.67**	306 ± 34	0.82 ± 0.05	7k params	13 s
1D-CNN	default	30 / 125	19.73	16.99 ± 0.61	529 ± 95	0.78 ± 0.05	74k params	10 s
1D-CNN	tuned ✓ final	40 / 135	16.36	**14.90 ± 0.55**	394 ± 70	0.92 ± 0.03	46k params	7 s

Means ± standard deviations over five seeds; the split and preprocessing are fixed, so the spread reflects model initialisation and sampling only. The linear model is deterministic. Train time is per seed on a laptop CPU.

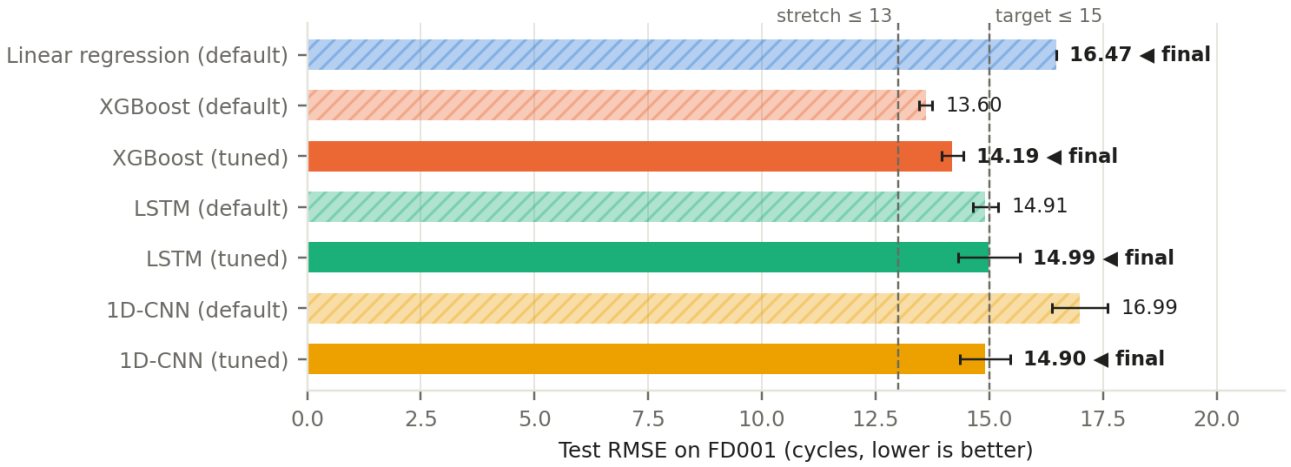


Figure 1. Test RMSE by model, mean ± std over five seeds. Solid bars: tuned; hatched: default parameters. ◀ marks the validation-selected final configuration.

4.2 Tuned hyperparameters

Model	Window / cap	Selected hyperparameters	Best val RMSE	Trials (pruned)	Tuning time
XGBoost	40 / 135	depth 5, learning rate 0.042, subsample 0.56, col. sample 0.44, min child weight 5, λ 4.1	14.81	40 (0)	2 min
LSTM	40 / 125	1 layer × 32 units, dropout 0.26, lr 0.0013, batch 256	14.42	25 (5)	14 min
1D-CNN	40 / 135	4 conv layers × 16 channels, kernel 5, dropout 0.13, lr 0.0009, batch 256	15.56	25 (5)	6 min

Every family chose the longest window offered (40 cycles), which suggests longer windows are worth exploring. The tuned LSTM is much smaller (7k parameters, vs 56k for the default) and trains faster.

4.3 Where the errors are

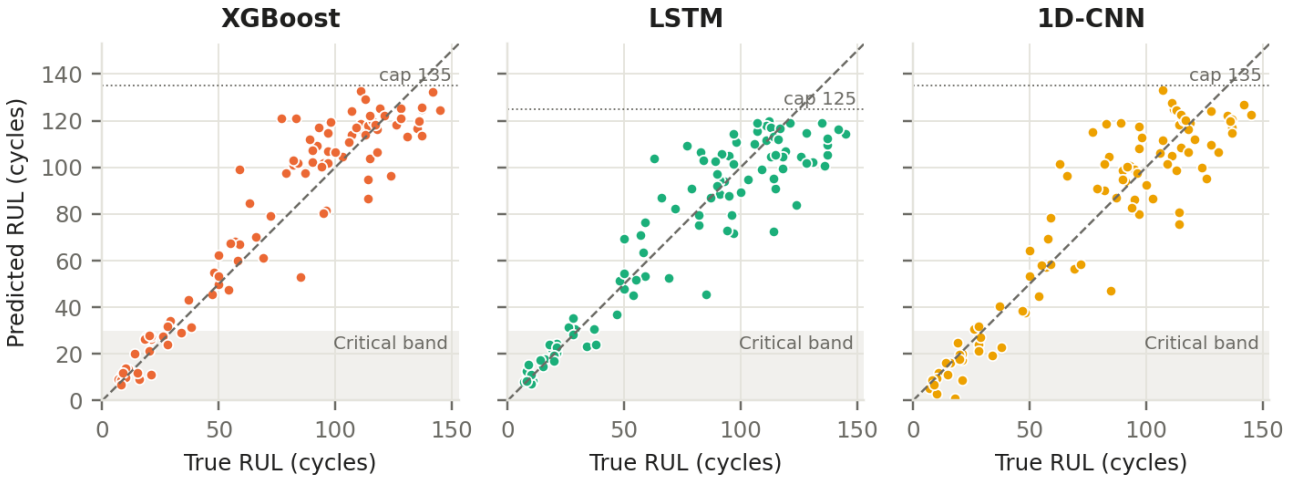


Figure 2. Predicted vs true RUL on the FD001 test set (seed 0, final configurations). Dashed: perfect prediction; dotted: the model's RUL cap; shaded: the Critical band.

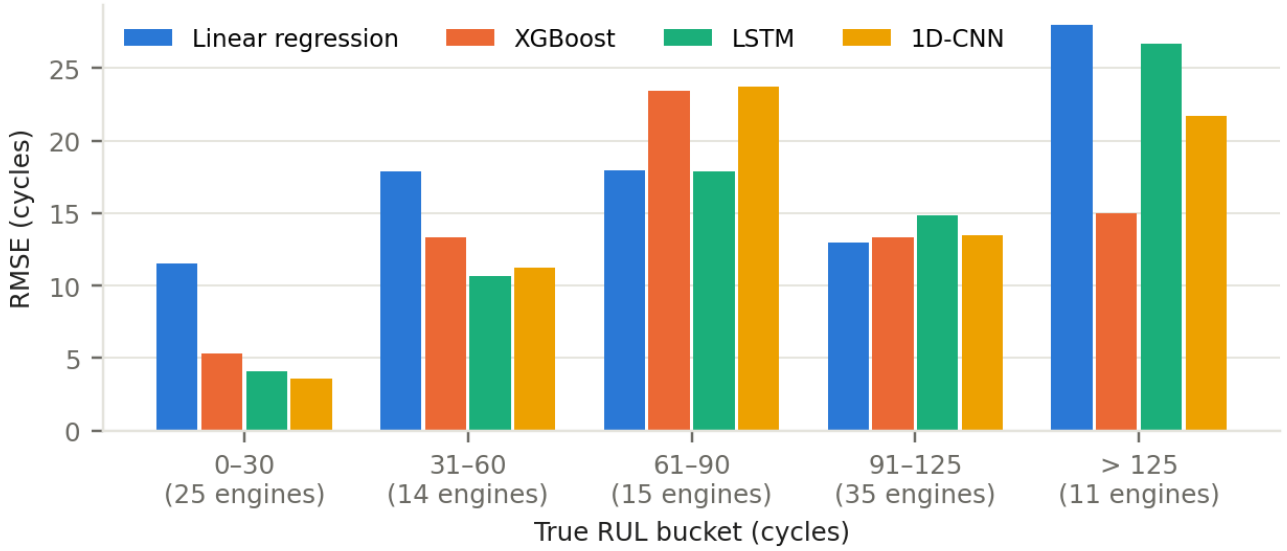


Figure 3. Test RMSE by true-RUL bucket, mean over five seeds (final configurations).

- **Most accurate where it matters.** Below 30 cycles the deep models err by about 4 cycles and XGBoost by 5. No engine with true RUL ≤ 30 is ever predicted above 60 (Healthy) by any model or seed.
- **Mid-life engines are predicted late.** In the 61–90 bucket, predictions run 6–14 cycles too high on average. This is the costliest error for the NASA score and for planning.
- **The cap limits long-life engines.** The 11 test engines with true RUL above 125 are under-predicted by 14–26 cycles. This is by design and harmless for planning, but it inflates RMSE. XGBoost's higher cap (135) reduces it the most.

4.4 Why the deep models do not beat XGBoost (Phase 3 exit criterion)

- **The gap is small.** 1D-CNN is 0.7 cycles behind XGBoost, about one seed standard deviation (0.55). The deep models lead on the planner-facing metrics: best NASA score (LSTM, 306) and best critical recall (1D-CNN, 92%).
- **The data is small.** There are only 80 training engines. Their overlapping windows are highly correlated, so the effective sample size is much smaller than the ~14k windows suggest. Engineered slope and level features hand XGBoost the degradation trend that a sequence model must learn from scratch.
- **Model selection is noisy.** Tuning cut validation RMSE by 2–3 cycles for every family, but test RMSE changed by XGBoost +0.6, LSTM +0.1, 1D-CNN -2.1. With 20 validation engines, the chosen configurations partly fit validation noise.
- **The tuning budget was modest.** Deep models had 25 trials capped at 40 epochs, against XGBoost's 40 cheap trials.
- **Published 11–13 RMSE figures use varied protocols**, for example capped test truth or best-of-runs reporting. They are not directly comparable to a five-seed mean scored against uncapped truth, and still need verifying (PRD §2).

5. Exploratory findings that shaped the pipeline (Phase 1)

- **Flat sensors.** 7 of 21 sensors carry no signal on FD001 (s1, s5, s6, s10, s16, s18, s19). Selection is per subset (s10 matters in FD003), so a trend-strength rule replaces a fixed list.
- **Operating conditions.** In FD002/FD004, per-condition normalisation lifts sensor–RUL correlation from $|\rho|$ ≈ 0.1 to ≈ 0.7.
- **Fault modes.** In FD003/FD004, five sensors move in opposite directions under the two fault modes. This motivates per-engine slope features and gives a signal for fault classification (FR-13).
- **Data correction.** FD004 has 249 training and 248 test engines; the dataset readme swaps these counts.

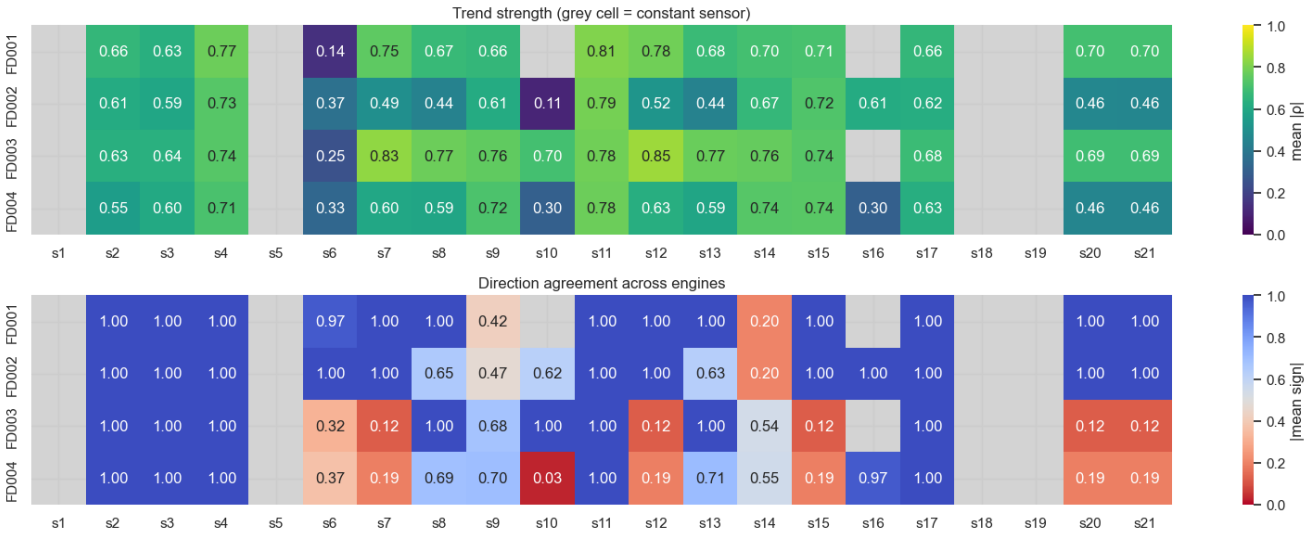


Figure 4. Sensor trend strength (top) and direction agreement across engines (bottom) per subset, after per-condition normalisation. Grey = constant sensor.

6. Work completed

Component	Purpose
data.py	Load FD001–FD004, name columns, RUL labels with configurable cap (FR-1).
preprocess.py	k-means operating conditions, per-condition z-scores, trend-strength sensor selection (FR-2).
split.py	Engine-level train/validation split (FR-3) and test-like truncation of validation engines.
windows.py, features.py	Sliding and last windows with front-padding; six statistics per sensor per window.
models/baselines.py, models/deep.py	Linear, XGBoost, LSTM and 1D-CNN behind one fit/predict interface (FR-4, FR-5).
metrics.py	RMSE, NASA score, critical-engine recall (FR-6).
pipeline.py, train.py, tune.py	Shared preparation; CLI that trains over seeds, logs to MLflow and results.csv, and saves per-engine predictions; Optuna tuning that never loads the test set.
tests/	35 unit tests, all passing: data 14, metrics 3, preprocess 6, split 6, windows 6.
configs/default.yaml	Every setting in one place: caps, windows, seeds, split, threshold, model defaults, tuning budget.

7. Next steps

Phase	Weeks	Scope	Exit criterion
4 · Robustness	7–8	Run FD002–FD004. Switch tuning to engine-level k-fold cross-validation, decided now, before any of those test scores exist. Add uncertainty ranges (MC dropout / quantile XGBoost), risk tiers and SHAP.	RMSE on all four subsets; ranges and explanations working
5 · Product	9–10	Export the best model and preprocessor; FastAPI /predict and /fleet; three-screen Streamlit dashboard.	All screens working locally
6 · Showcase	11–12	Deployment, README, course report, demo video, write-up.	Public demo live

Candidate improvements, to be judged on validation only: windows longer than 40 cycles; an asymmetric loss that penalises late predictions (the 61–90 bucket); averaging predictions across seeds.

8. Limitations

- The data is simulated, and failure is a threshold crossing rather than a removal decision. There is no cost or schedule data, so maintenance value is argued, not measured.
- Results so far cover FD001 only, the easiest subset.
- Each test set has 100 engines and only 25 with true RUL ≤ 30, so each critical engine moves recall by 4% and differences of a few tenths of a cycle in RMSE are not meaningful.
- The validation set has only 20 engines, which limits how reliably configurations can be ranked (Section 4.4).